

Reviewer Pack — Halton Star-Discrepancy of Clause-Sign Embedding Bounds DPLL...

Entry c4e28e9508f6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 03:49:44 UTC

Halton Star-Discrepancy of Clause-Sign Embedding Bounds DPLL Leaves

Entry ID: c4e28e9508f6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 03:49:44 UTC

1. Conjecture

Field A (mathematical branch): Quasi-Monte Carlo low-discrepancy sequence theory (Halton/van der Corput star-discrepancy) **Field B** (complexity object): DPLL search-tree leaf count on random 3-CNF instances

Statement:

Embed each clause C_i of a 3-CNF F over n variables as a point p_i in $[0,1]^3$ by writing the three signed-literal indices (signvar) in base-2 reversed (van der Corput) coordinates, then let $D(F)$ be the empirical star-discrepancy of $\{p_i\}$ relative to the uniform measure on $[0,1]^3$. Conjecture: for unsatisfiable random 3-CNF at clause density $\alpha=4.267$, the DPLL leaf count $L(F)$ satisfies $\log_2 L(F) \geq c \cdot n \cdot D(F) - O(\log n)$ with $c = 1/2$, and equivalently $\log_2 L(F) \leq C \cdot n \cdot D(F) + O(\log n)$ with $C = 4$. A single instance with $\log_2 L(F) < 0.4 \cdot n \cdot D(F) - 5$ ($n \leq 20$) falsifies the lower bound.

Rationale (proposer's reasoning):

Star-discrepancy measures how 'evenly' clause-signatures fill literal-space; Chazelle/Matousek showed star-discrepancy controls combinatorial concentration phenomena that mirror branching irregularity. Highly clustered clause-embeddings (low discrepancy in sub-boxes) should correspond to local refutations exploitable by short DPLL trees, while highly dispersed sets force the splitter to branch globally. The bridge is constructive (just digit-reversal + brute search-tree count) and bypasses natural-proofs since star-discrepancy is not a pseudorandom-friendly invariant.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fb449892ce6181e0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 200 unsatisfiable random 3-CNFs per n in $\{10,12,14,16,18,20\}$ at $\alpha=4.267$ (1200 seeds total), per-instance compute $D(F)$ by exact enumeration and DPLL leaf count

$L(F)$. *SUPPORTED* iff $\geq 85\%$ of all 1200 seeds satisfy $0.5nD(F) - 5 \leq \log_2 L(F) \leq 4nD(F) + 5$; *FALSIFIED* iff any single seed yields $\log_2 L(F) < 0.4nD(F) - 5$ with $n \leq 20$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.86	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Halton van der Corput star-discrepancy SAT 3-CNF clause embedding - low-discrepancy sequence DPLL search tree size random 3-SAT - quasi-Monte Carlo discrepancy resolution complexity unsatisfiable 3-CNF leaves

Top relevant hits considered: - [<http://arxiv.org/abs/1209.3611v2>] A van der Corput-type algorithm for LS-sequences of points - [<http://arxiv.org/abs/2411.14979v2>] Fundamentals and applications of Van der Waals magnets in magnon spintronics - [<http://arxiv.org/abs/1908.01624v1>] Learned Clause Minimization in Parallel SAT Solvers - [<http://arxiv.org/abs/1402.4455v1>] Symbiosis of Search and Heuristics for Random 3-SAT - [<http://arxiv.org/abs/cs/0506069v1>] A generating function method for the average-case analysis of DPLL - [<http://arxiv.org/abs/1908.05361v2>] Placing quantified variants of 3-SAT and Not-All-Equal 3-SAT in the polynomial hierarchy - [<http://arxiv.org/abs/1902.04347v4>] A Multilevel Monte Carlo Asymptotic-Preserving Particle Method for Kinetic Equations in the Diffusion Limit - [<http://arxiv.org/abs/physics/9611010v1>] Quasi-Monte Carlo, Discrepancies and Error Estimates

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=180.0s

5.1 Generated Python source

```
import random
import math
import json
from itertools import product

def generate_3cnf(n, alpha):
    m = int(alpha * n * (n - 1) / 2)
    clauses = set()
    while len(clauses) < m:
        clause = tuple(sorted(random.sample(range(1, n + 1), 3)))
        if random.choice([True, False]):
            clause = (-x for x in clause)
        clauses.add(tuple(abs(x) for x in clause))
    return clauses

def van_der_corput_base2(num, base=2):
```

```

result = []
while num > 0:
    result.append(int(num % base))
    num //= base
return result[::-1]

def embed_clause(clause, n):
    points = []
    for literal in clause:
        var = abs(literal)
        sign = -1 if literal < 0 else 1
        digits = van_der_corput_base2(var + n, base=2)
        point = [sign * d / (2 ** len(digits)) for d in digits]
        points.append(point)
    return points

def star_discrepancy(points):
    m = len(points)
    if m == 0:
        return 0
    min_d = float('inf')
    max_d = -float('inf')
    for i, j in product(range(m), repeat=2):
        d = sum(abs(p[i] - q[j]) for p, q in zip(points, points))
        if d < min_d:
            min_d = d
        if d > max_d:
            max_d = d
    return (max_d - min_d) / m

def dpll(clause_set):
    def backtrack(assignment, clause_set):
        if not clause_set:
            return True
        for literal in clause_set[0]:
            new_assignment = assignment[:]
            if literal > 0:
                new_assignment[literal - 1] = True
            else:
                new_assignment[-literal - 1] = False
            if backtrack(new_assignment, [c for c in clause_set if not any(l in c for l in new_assignment)]):
                return True
        return False
    assignment = [False] * (max(clause_set) + 1)
    return backtrack(assignment, clause_set)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [10, 12, 14, 16, 18, 20]
    results = []

    for n in n_values:
        alpha = 4.267
        clauses = generate_3cnf(n, alpha)
        points = sum((embed_clause(clause, n) for clause in clauses), [])

```

```

    if not dpll(clauses):
        D_F = star_discrepancy(points)
        L_F = len(dpll(clauses))

        results.append({
            "n": n,
            "D_F": D_F,
            "L_F": L_F
        })

total_dF = sum(result["D_F"] for result in results)
total_LF = sum(result["L_F"] for result in results)
mean_DF = total_dF / len(results)
mean_LF = total_LF / len(results)

support_fraction = sum(0.5 * n * result["D_F"] - 5 <= math.log2(result["L_F"]) <= 4 * n * resu

if support_fraction >= 0.85:
    return {
        "metric_name": "support_fraction",
        "metric_value": support_fraction,
        "instances_tested": len(results),
        "conjecture_holds": True,
        "counterexample": ""
    }
else:
    for result in results:
        if result["n"] <= 20 and math.log2(result["L_F"]) < 0.4 * result["n"] * result["D_F"]:
            return {
                "metric_name": "support_fraction",
                "metric_value": support_fraction,
                "instances_tested": len(results),
                "conjecture_holds": False,
                "counterexample": f"n={result['n']}, D_F={result['D_F']}, L_F={result['L_F']}"
            }
    return {
        "metric_name": "support_fraction",
        "metric_value": support_fraction,
        "instances_tested": len(results),
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(json.dumps({"TRIAL": {"seed": seed, **result}}))

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 180s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out after 180s (returncode=124) without producing any data, so neither the support condition nor the falsification condition could be evaluated. | next: Reduce the workload (e.g., fewer seeds per n or cap n at 16) and/or add per-instance timeouts so the 1200-seed sweep can complete within the budget.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	19691
2	preregistration	claude_max	opus	0	0	5609
3	novelty	claude_max	opus	0	0	3757
4	novelty	claude_max	opus	0	0	9061
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19578
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12319
7	judge	claude_max	opus	0	0	3917

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 73932 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in *research/audit/c4e28e9508f6.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/c4e28e9508f6.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/c4e28e9508f6.tar.gz* (if generated)